



Javier Urbaneja Benitez

25/09/2025

STAGE 1

- ① The file "echo-server.c" doesn't compile at first and shows the following error:
- "struct sockaddr_in has no member named 'sin_port'; did you mean 'sin_port'?"

I noticed that the error was in the field of the variable "serv_addr" and I checked the error to write it correctly.

- ② The file "echo-server.c" doesn't run at first:
- "bind: Permission denied"

I noticed that the error was related to the port number because "bind()" fails when trying to associate the socket to the port.

The port number was set to 80 which is a privileged port. I changed it to an unprivileged port (12345) and then it runs correctly.



- ③ If I send to the server more than 1024 bytes, the client shows the following error:
- "read: connection reset by peer"

this occurs because the server only reads at most 1024 bytes and then closes the connection (it only calls "read()" function once and then stops reading), so the client keeps waiting until the server echoes the number of bytes that it sent and when it tries to read, it notices that the server has closed (it doesn't echoed all bytes).

I made a loop at "echo-server" that keeps reading until it reads the entire message in (chunks of) chunks of 1024 bytes and now it works.

STAGE 2

The time generally increases as the message size increases, but it doesn't grow exactly in proportion to the size because each connection is divided in chunks of 1024 bytes.

we can see that in the first and second test: ~~As~~ they both fit in only one chunk, their average time is similar even if the second ~~(test)~~ (th) test is 100 times bigger than the second.

STAGE 3

① The client sends 1025 bytes and it wait until having those bytes ~~to~~ echoes. As I stop the server when it only echoed 1024 bytes, the client tries to receive the ~~the~~ missing byte and throws an error calling "read()" (there's nothing to read).



② The client tries to connect to the server at a different port that the server is, it throws an error calling "connect()" because the client can't find the server.

③ As the server is sequential, if I run two different messages from different clients, the ~~(second)~~ client that sent it later waits until the first one finish.

In the test, I send 1025 bytes from both clients ~~(and the)~~, ~~(the 1)~~ both could connect to the server and send their message, but as the server is sequential, it put the second ~~(message)~~ connection in queue and processes the first one. The second client waits in read loop until the server ~~(replays)~~ answers.